# The Impact of Input Error on the Scheduling of Task Graphs with Imprecise Computations in Heterogeneous Distributed Real-Time Systems

Georgios L. Stavrinides and Helen D. Karatza

Department of Informatics,
Aristotle University of Thessaloniki,
54124 Thessaloniki, Greece
{gstavrin,karatza}@csd.auth.gr

**Abstract.** In real-time systems, it is often more desirable for a job to produce an approximate, imprecise result by its deadline than to produce a precise result late. In this paper, we evaluate by simulation the performance of a heterogeneous distributed real-time system, where composite jobs with end-to-end deadlines are scheduled dynamically as they arrive in the system, utilizing imprecise computations. Each job is a directed acyclic graph of component tasks, where the output data of a task may be used as input by another task. In case the input data of a component task are imprecise, the processing time of the task is extended, in order to correct the error and produce a result of acceptable quality. The impact of input error on the system performance is investigated under various workloads and input error limits.

**Keywords:** heterogeneous distributed real-time systems; task graphs; imprecise computations; input error; simulation.

## 1 Introduction

Due to the inherent need of many scientific and commercial applications for high quality results within strict timing constraints, distributed real-time systems have become increasingly important. They are employed in many aspects of our daily life, such as the control of nuclear power plants, financial markets, radar systems, telecommunication networks, intensive care monitoring and multimedia applications. In a real-time system, the jobs have *deadlines* that must be met. The correctness of the system does not depend only on the computational results, but also on the time at which the results are produced. In case a real-time job fails to meet its deadline, its results will be useless [1].

As a more cost-effective solution, many of today's distributed real-time systems are heterogeneous. That is, they comprise different types of processors and networks. The major challenge in such a real-time heterogeneous environment, is the employment of an effective scheduling technique, in order to guarantee that every real-time job will produce high quality results within the imposed timing

K. Al-Begain et al. (Eds.): ASMTA 2011, LNCS 6751, pp. 273–287, 2011.
© Springer-Verlag Berlin Heidelberg 2011

constraints. The scheduling algorithm is responsible for the allocation of jobs to processors and determines the order in which jobs will be executed.

In distributed real-time systems, a job may be modelled as a *directed acyclic graph* (DAG or task graph), where the nodes represent the component tasks of the job and the edges represent the data dependencies between the tasks. Due to the precedence constraints among the component tasks, a task's output is used as input by other tasks of the job. Each job has an *end-to-end* deadline that must be met. That is, the tasks of a job do not have any specific individual deadlines. In order for a task to start execution, all of its predecessor tasks must have been completed. A task with no predecessors is called an *entry* task, whereas a task with no successors is called an *exit* task. The immediate predecessors of a task are called *parents* of the particular task, while the immediate successors of a task are called *children* of the particular task. In order for a job to meet its deadline, i.e. to be *guaranteed*, all of its exit tasks must finish execution before the job's deadline is reached.

### 1.1 Related Work

Based on the observation that in real-time systems it is often more desirable for a job to produce an approximate result by its deadline, than to produce a precise result late, Lin et al. proposed the *imprecise computations* technique [2]. According to this method, the execution of a real-time job is allowed to return intermediate (*imprecise*) results of poorer, but still acceptable quality, when the deadline of the job cannot be met. To achieve this, it is assumed that every component task in a job is *monotone*. That is, the quality of its results is improved as more time is spent to produce them. If a monotone task is fully completed, then its results are precise.

It is often assumed that each monotone task of a job consists of a *mandatory part*, followed by an *optional part* [3]. In order for a task to produce an acceptable result, its mandatory part must be completed. The optional part refines the result produced by the mandatory part. *Input error* may affect the processing time of a task. Specifically, when the input data of a task are imprecise, the processing time of the task is extended, in order to correct the error and produce a result of acceptable quality [4]. A job is completed only when all of its tasks have completed *at least* their mandatory part, before the job's deadline.

Because of its key importance, scheduling in distributed real-time systems has been studied extensively in the literature [5], [6], [7]. Among the real-time scheduling policies that have been proposed, the *Earliest Deadline First* (EDF) algorithm is the most commonly used [8]. According to this technique, the job with the earliest deadline has the highest priority for scheduling. Heuristics for task graph scheduling have been proposed by many authors [9], [10]. The imprecise computations model has been studied and extended in the literature. In [11], Feng and Liu investigate the impact of input error on the static scheduling of a single, linear chain of tasks with an end-to-end deadline on a single processor. That is, each component task is considered to have only one parent task. On the other hand, in [12], Haweet et al. examine the static scheduling of a single task

graph with an end-to-end deadline on a single processor, but without taking into account the effects of input error on the execution time of the component tasks.

### 1.2 Our Approach

In this paper, we evaluate by simulation the performance of a heterogeneous distributed real-time system, where task graphs with end-to-end deadlines are scheduled dynamically as they arrive in the system, utilizing imprecise computations. The output data of a component task in a DAG may be used as input by more than one task, according to the data dependencies in the graph. In case the input data of a component task are imprecise, the processing time of the task is extended, in order to correct the error and produce a result of acceptable quality. The minimum result precision that is acceptable from a task is determined by a predefined result precision threshold. Furthermore, each component task has an input error limit, beyond of which the error in its input cannot be corrected and an acceptable result cannot be produced, no matter how long the task executes.

The work presented in this paper differs from our previous work in [7] mainly in two aspects: the system model and the imprecise computations model. Specifically, in this paper we consider a heterogeneous environment instead of a homogeneous system. Moreover, in contrast to [7] where the parent tasks of a task can all omit their whole optional part, in this case study there is a limit on the allowed input error of a task. Therefore, the fraction of the optional part that each parent task can discard is restricted. Our foremost goal is to guarantee that all jobs that arrive in the system will meet their deadlines, providing results as precise as possible. The main focus of this paper is the investigation of the impact of input error on the system performance, under various workloads and input error limits.

The remainder of this paper is organized as follows: Section 2 gives a description of the system, the workload and the imprecise computations model under study, Section 3 describes the on-line strategy used to schedule the task graphs that arrive in the system and Section 4 presents and analyzes the simulation results. Finally, Section 5 concludes this paper, providing suggestions for further research.

## 2 System and Workload Model

The target system is considered to be a heterogeneous distributed real-time system, consisting of a set $P$ of $q$ processors, each serving its own queue (memory). Each processor $p_i$ is characterized by an execution rate $\mu_i$. Since the system is heterogeneous, the processor execution rates may vary. It is assumed that the processors in the system are fully connected, that the communication is contention-free and that a processor can receive and send data to other processors simultaneously, even when it is executing a task. Thus, no attention is paid to the routing strategies used for communication. $N$ is the set of communication links in the system. The transfer rate between two processors $p_i$ and $p_j$ is

denoted by $\nu_{ij}$. The communication startup time of processor $p_i$ is taken into account in the value of the transfer rate $\nu_{ij}$. It is assumed that there is only one bidirectional communication link between any pair of processors. That is, $\nu_{ij} = \nu_{ji}$ for any pair of processors $p_i$ and $p_j$. As in the case of the processor execution rates, the data transfer rates between the processors may vary as well.

Real-time jobs arrive in the system in a Poisson stream with rate $\lambda$. Each job that arrives in the system, is a directed acyclic graph $G = (V, E)$, where $V$ is the set of the $a$ nodes of the graph and $E$ is the set of the $b$ directed edges between the nodes of the graph. The terms *job*, *task graph* and *DAG* are used interchangeably in this paper. The same applies for the terms *task* and *node*. Each node in a job represents a component task $n_i$ of the job, whereas a directed edge $e_{ij}$ between two tasks $n_i$ and $n_j$ represents a message that must be transmitted from task $n_i$ to task $n_j$. A job that arrives in the system may have one or more entry tasks and one or more exit tasks. A task can start execution only if it has received its required input data from all of its parent tasks. It is assumed that no additional data are required during a task's execution and that the output data of a task are available only after the task's completion. The execution of the tasks is assumed to be non-preemptive.

Each node $n_i$ in a job has a weight $w_i$ which denotes its *computational volume*, i.e. the amount of computational operations needed to be executed. The *computational cost* of the task $n_i$ on a processor $p_j$ is given by

$$Comp(n_i, p_j) = w_i/\mu_j \tag{1}$$

where $\mu_j$ is the execution rate of processor $p_j$. The *average computational cost* of the task $n_i$ over all of the processors in the system, is defined as

$$\overline{Comp(n_i)} = \frac{\sum_{p_j \in P} Comp(n_i, p_j)}{q} \tag{2}$$

where $P$ is the set of processors and $q$ is the number of processors in the system.

Each edge $e_{ij}$ between two tasks $n_i$ and $n_j$ has a weight $c_{ij}$ which represents its *communication volume*, i.e. the amount of data needed to be transferred. The *communication cost* of the edge $e_{ij}$, which is incurred when transferring data from task $n_i$ to task $n_j$, scheduled on processors $p_m$ and $p_n$ respectively, is defined as

$$Comm\left((n_i, p_m), (n_j, p_n)\right) = c_{ij}/\nu_{mn} \tag{3}$$

where $\nu_{mn}$ is the data transfer rate of the communication link between the processors $p_m$ and $p_n$. When both tasks $n_i$ and $n_j$ are scheduled on the same processor, the communication cost of the edge $e_{ij}$ becomes zero, since it is considered negligible when compared with interprocessor communication costs. The *average communication cost* of the edge $e_{ij}$ over all of the communication links in the system, is given by

$$\overline{Comm(e_{ij})} = \frac{\sum_{\nu_{mn} \in N} Comm\left((n_i, p_m), (n_j, p_n)\right)}{q(q-1)/2} \tag{4}$$

where $N$ is the set of communication links and $q(q-1)/2$ is the number of communication links in the system.

Each component task $n_i$ of a job is assigned a *priority value* $PV_i$, which determines its priority for scheduling. The *level* $L_i$ of a task is the length of the longest path from the particular task to an exit task. The length of a path in the graph is the sum of the computational and communication costs of all of the tasks and edges, respectively, on the path. The level of task $n_i$ is given by

$$L_i = \overline{Comp(n_i)} + \max_{n_j \in S_i} \left\{ \overline{Comm(e_{ij})} + L_j \right\} \tag{5}$$

where $S_i$ is the set of the child tasks of task $n_i$. The level of an exit task is equal to the task's computational cost. The *critical path length* $CPL$ of a job is the length of the longest path from an entry task to an exit task in the graph. The *relative end-to-end deadline* $RD$ of a job is the interval between the job's arrival time $A$ and its *absolute end-to-end deadline* $D$ (measured from time zero). That is,

$$RD = D - A. \tag{6}$$

The *communication to computation ratio* $CCR$ of a job is the ratio of its average communication cost to its average computational cost on the target system and is defined as

$$CCR = \frac{\sum_{e_{ij} \in E} \overline{Comm(e_{ij})}}{\sum_{n_i \in V} \overline{Comp(n_i)}} \tag{7}$$

where $V$ and $E$ are the sets of the nodes and the edges of the job respectively. The value of $CCR$ indicates whether the DAG is communication intensive, computationally intensive or moderate. An example task graph with 2 entry tasks and 4 exit tasks, is illustrated in Fig. 1.

In this paper, it is assumed that there is a centralized scheduler that has full knowledge of the system. When a task graph arrives in the system, all of its tasks first enter a *global waiting queue*. In this queue, the unscheduled tasks of all the arrived jobs wait until they get ready to be scheduled. A task is ready to be scheduled when it has no predecessors (i.e. if it is an entry task) or when all of its parent tasks have finished execution. Thus, there are no data dependencies between tasks that are ready for scheduling. Once one or more tasks become ready to be scheduled, something which occurs after a job arrival or after a task completion, the scheduler is invoked. The scheduler allocates a processor to each ready task in the global waiting queue and determines the task's position in the processor's respective *local waiting queue*, as explained in Section 3. Clearly, a local waiting queue may contain tasks of different jobs. We assume that task migrations are not permitted between the processors and that software and hardware failures do not occur during the execution of a task.

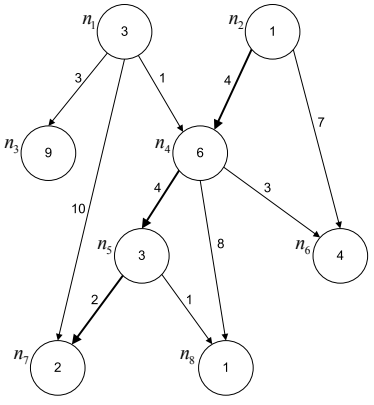



**Fig. 1.** An example task graph. The number in each node denotes the task's average computational cost, whereas the number on each edge denotes the average communication cost of the edge. The level of task $n_2$ is the highest in the graph and it is equal to $L_2 = 22$. The DAG's critical path length is equal to the highest entry task level in the graph, $L_2$. That is, $CPL = 22$. The edges on the critical path of the graph are shown with thick arrows.

### 2.1 Imprecise Computations Model

It is assumed that every job that arrives in the system consists of component tasks that are *monotone*. The computational volume $w_i$ of each component task $n_i$ consists of a *mandatory part* $mp_i$, followed by an *optional part* $op_i$. That is,

$$w_i = mp_i + op_i \tag{8}$$

where $0 < mp_i < w_i$. A task is completed when *at least* its mandatory part has been processed. That is, a task can either complete its optional part entirely, partially or it may skip its whole optional part. When a task completes only a part of its optional part (or when its entire optional part is omitted), there is error in its results (*output error*) and therefore its results are *imprecise*. If the particular task is not an exit task in the graph, then its output data will be used as input by its child tasks. Therefore, the child tasks of the particular task will have *input error*. The output error of a task $n_i$ is given by

$$OE_i = \frac{op_i - \tau_i}{op_i} \tag{9}$$

where $\tau_i$ is the processed part of the optional part $op_i$ of the task. Thus, $op_i - \tau_i$ gives the amount of discarded work of task $n_i$. Since $0 \leq \tau \leq op_i$, from (9), it follows that $0 \leq OE_i \leq 1$.

The input error of a task $n_i$ is equal to the total output error of the task's parent tasks. That is,

$$IE_i = \sum_{n_j \in Z_i} OE_j \tag{10}$$

where $Z_i$ is the set of the parent tasks of task $n_i$. If a task has input error, then there is an impact on its execution time. Specifically, we assume that its mandatory part is extended, since more computations are required by the task to correct the error and produce an acceptable result. However, it is assumed that the optional part of the task is not affected by its input error. For instance, in weather forecasting systems, there is a satellite data collection task $n_1$ and a data visualization task $n_2$, which produces the satellite images. Assume that the processing time of the mandatory part of task $n_2$ is the time required by the task to process and visualize the data provided by task $n_1$, in order to produce an image of an acceptable standard. The processing time of the optional part of task $n_2$ is the time required by the task to enhance the quality of the image produced by its mandatory part. If the data provided to task $n_2$ by task $n_1$ are imprecise (e.g. due to noise during the satellite transmission), then task $n_2$ will require more time to process the data and produce an image of the acceptable quality. That is, the mandatory part of task $n_2$ is extended due to input error. The optional part of task $n_2$ remains the same. It is assumed that the size of the data produced by a task – and thus the communication cost of the task's outgoing messages to its child tasks – is independent of the precision of the data.

The *mandatory part extension* of a task $n_i$ is given by

$$mpe_i = mp_i \cdot IE_i \tag{11}$$

where $mp_i$ is the mandatory part and $IE_i$ is the input error of the task $n_i$ respectively. A task $n_i$ cannot always eliminate its input error and produce a precise result by just extending its mandatory part. That can only happen when its input error $IE_i$ is less than or equal to its *input error limit* $IEL_i$ ($0 \leq IE_i \leq IEL_i$). Otherwise, if the task's input error exceeds the input error limit of the task, then the task can never give an acceptable result, no matter how long it executes.

The minimum result precision that is acceptable from a task is determined by a predefined *result precision threshold* which is fixed for all tasks. A task $n_i$ provides results of the minimum quality when only its mandatory part $mp_i$ is processed and its whole optional part $op_i$ is omitted. The relation between the result precision threshold $RPT$ and the mandatory part $mp_i$ of a task is given by

$$mp_i = RPT \cdot w_i \tag{12}$$

where $w_i$ is the computational volume of task $n_i$. Consequently, the *result precision* of a task $n_i$ is defined as

$$RP_i = RPT + (1 - RPT)(1 - OE_i) \tag{13}$$

where $OE_i$ is the output error of the task. The result precision of every completed task is $RPT \leq RP \leq 1$. When a task $n_i$ completes both its mandatory part $mp_i$ and its optional part $op_i$, it produces precise results (i.e. its output error is $OE_i = 0$) and therefore its result precision is $RP_i = 1$. On the other hand, if a task completes only its mandatory part $mp_i$, then it produces imprecise results

of the minimum acceptable quality (i.e. its output error is $OE_i = 1$) and thus its result precision is $RP_i = RPT_i$. The result precision of a job ultimately depends on the precision of the results of its exit tasks only. Therefore, the result precision $RP_k$ of a job $G_k$ is considered to be equal to the average result precision of $G_k$'s exit tasks.

A task $n_i$ which is in service on a processor $p_m$ can be aborted prematurely, if the following conditions hold:

1. There is a task waiting for service at the head of $p_m$'s local waiting queue.
2. Task $n_i$ is not an exit task of its job.
3. Task $n_i$ has completed its mandatory part.
4. The potential output error of task $n_i$ (that is, its output error in case it is aborted at the current time instant) is less than or equal to the minimum difference between the input error limit and the current input error of its child tasks. That is,

$$OE_i \leq \min_{n_j \in S_i} \{IEL_j - IE_j\} \tag{14}$$

   where $OE_i$ and $S_i$ are the potential output error and the set of child tasks of task $n_i$ respectively, whereas $IEL_j$ and $IE_j$ are the input error limit and the current input error of a child task $n_j$ in $S_i$, respectively.
5. The processing time saved by skipping the unprocessed part of the optional part of task $n_i$ is greater than or equal to the total average additional processing time imposed to $n_i$'s child tasks, due to its potential output error. That is,

$$(op_i - \tau_i)/\mu_m \geq \sum_{n_j \in S_i} \frac{\sum_{p_n \in P} (OE_i \cdot mp_j)/\mu_n}{q} \tag{15}$$

   where $op_i$, $\tau_i$, $S_i$ and $OE_i$ are the optional part, the processed part of the optional part, the set of child tasks and the potential output error of the task in service $n_i$, $\mu_m$ is the execution rate of the processor $p_m$ of the task in service, $mp_j$ is the mandatory part of a child task $n_j$ in $S_i$, $P$ is the set of processors in the system, $q$ is the number of processors in the system and $\mu_n$ is the execution rate of a processor $p_n$ in $P$. Note that for each child task $n_j$ of the task in service $n_i$, the average of its imposed additional processing time is considered. This is due to the fact that all of the child tasks of the task in service are still unscheduled and thus not assigned to a particular processor.

A job is completed if all of its tasks are completed before the job's deadline. In the case where an uncompleted job's deadline is reached, then the job is not always considered lost. Specifically, if the job's last tasks remaining to be completed are exit tasks and all of them have completed their mandatory part at the time of their job's deadline, then the tasks, and thus the job, are considered completed. In any other case the job is lost.

## 3 Scheduling Strategy

The ready tasks in the global waiting queue are scheduled according to a list scheduling heuristic, which consists of a task selection phase and a processor selection phase, as described below.

### 3.1 Task Selection Phase

Each ready task is assigned a priority value according to the Earliest Deadline First (EDF) policy. That is, the priority value $PV_i$ of a component task $n_i$ is considered to be equal to the absolute deadline $D$ of its job ($PV_i = D$). Subsequently, the tasks are arranged in a list according to their assigned priorities. The task with the highest priority for scheduling is the one with the smallest priority value (i.e. the one with the earliest end-to-end deadline). If two or more tasks have the same priority value, then they are arranged in descending order of average computational costs.

### 3.2 Processor Selection Phase

Once a ready task is selected by the scheduler, it is allocated to the processor which can provide it with the earliest *estimated start time EST* (ties are broken randomly). The $EST$ of a ready task $n_i$ on a processor $p_j$ is given by the expression

$$EST(n_i, p_j) = \max\{T_{data}(n_i, p_j), T_{idle}(n_i, p_j)\} \tag{16}$$

where $T_{data}(n_i, p_j)$ is the time interval after which all input data of task $n_i$ will be available on processor $p_j$, whereas $T_{idle}(n_i, p_j)$ is the time interval after which the task $n_i$ will reach the head of $p_j$'s local waiting queue and $p_j$ will be idle and thus able to execute task $n_i$.

$T_{data}(n_i, p_j)$ is equal to the largest communication cost between $n_i$ and its parent tasks. If a parent task of $n_i$ was executed on processor $p_j$, we consider the respective communication cost to be equal to zero.

In order to calculate $T_{idle}(n_i, p_j)$, we first find the *potential position* of the ready task $n_i$ on processor $p_j$. That is, we find the position at which the ready task $n_i$ would be placed in the local waiting queue of processor $p_j$, if it was actually assigned to that particular processor. Consequently, if task $n_i$ would be placed at the head of $p_j$'s queue and $p_j$ is busy executing another task which cannot be aborted, then $T_{idle}(n_i, p_j)$ is equal to the remaining execution time of the task in service. Otherwise, if $p_j$ is idle, then $T_{idle}(n_i, p_j) = 0$. On the other hand, if $n_i$ would be placed at any other position in $p_j$'s queue, so that it would be preceded by a task $n_r$, then

$$T_{idle}(n_i, p_j) = EST(n_r, p_j) + Comp(n_r, p_j) \tag{17}$$

where $EST(n_r, p_j)$ and $Comp(n_r, p_j)$ are the estimated start time and the computational cost of the preceding task $n_r$ respectively. Once a ready task is scheduled to the selected processor, it may be required to wait in order to receive data

from other processors. Otherwise, the task is ready for processing and therefore it waits for its turn to start execution. It is assumed that the scheduling overhead is negligible.

## 4 Performance Evaluation

### 4.1 Performance Parameters

For the performance evaluation of the system in the framework described in the previous sections, we conducted a series of simulation runs using the independent replication method. Due to the complexity of the system and of the workload model under study, we implemented our own discrete-event simulation program in C++, tailored to the requirements of the specific case study.

The effects of input error on the performance of the system are investigated using the *overall system performance* metric, which takes into account not only the number of jobs that meet their deadlines (that is, the number of *guaranteed* jobs), but also the precision of the results of the guaranteed jobs. The overall system performance metric is given by

$$OSP = \frac{\sum_{G_k \in C} RP_k}{TNJA} \tag{18}$$

where $C$ is the set of all of the guaranteed (completed) jobs, $RP_k$ is the result precision of a completed job $G_k$ in $C$ and $TNJA$ is the total number of job arrivals. In the best case, all of the jobs that arrive in the system meet their deadlines and provide precise results and therefore, $OSP = 1$. In the worst case, all of the jobs that arrive in the system are lost and therefore, $OSP = 0$. Hence, the overall system performance $OSP$ is always in the range $[0, 1]$.

### 4.2 Task Graph Generation

In our simulation experiments, the jobs (DAGs) were generated randomly, using our own task graph generator, as explained in [7]. Each generated DAG is a *weakly connected* graph. That is, there is a path between any pair of tasks in the graph, without taking into account the direction of the edges. In the case where a graph has more than one task, there is at least one entry and one exit task in the graph. In order to generate a DAG, we first determine the number of tasks $a$ in the graph, which is uniformly distributed in the range $[1, MAX]$, where $MAX$ is the maximum number of tasks allowed in a graph. Subsequently, if $a > 1$, we determine the number of entry tasks $a_{entry}$ in the graph, which is uniformly distributed in the range $[1, a-1]$. We then create the first entry task.

Subsequently, the $a - a_{entry}$ non-entry tasks (which may be exit tasks - this is determined when the construction of the task graph is finally completed) of the graph are created, one by one. At each step, we create a task and we connect it to its parent tasks, that are randomly chosen from the already created tasks. The number of a task's parent tasks is uniformly distributed in the range $[1, f]$,

where $f$ is the number of tasks that have already been created. After creating the $a - a_{entry}$ non-entry tasks, we continue constructing the DAG, by creating the rest $a_{entry} - 1$ entry tasks, one by one. At each step, we create a task and we connect it to its child tasks, that are randomly chosen from the non-entry tasks already created. The number of a task's child tasks is uniformly distributed in the range $[1, a - a_{entry}]$. The uniform distribution is used for the generation of the task graphs, so that all DAG structures are equally probable.

### 4.3 Simulation Input Parameters

For our experiments, we used the simulation input parameters shown in Table 1. The *heterogeneity factor* $HF$ denotes the difference in the speeds of the processors, as well as in the transfer rates of the communication links in the system. The heterogeneity factor is in the range $0 \leq HF \leq 1$. In the case where $HF = 0$, the system is homogeneous. The execution rate of each processor in the system is uniformly distributed in the range $[\overline{\mu} \cdot (1 - HF/2), \overline{\mu} \cdot (1 + HF/2)]$, where $\overline{\mu}$ is the mean execution rate of the processors. The data transfer rate of each communication link in the system is uniformly distributed in the range $[\overline{\nu} \cdot (1 - HF/2), \overline{\nu} \cdot (1 + HF/2)]$, where $\overline{\nu}$ is the mean data transfer rate of the communication links. The computational volume of a task in a graph is exponential with mean $\overline{w}$, whereas the communication volume of an edge is exponential with mean $\overline{c}$. For a given $CCR$ and a mean computational volume $\overline{w}$, the mean communication volume $\overline{c}$ can be calculated from (7).

We investigated the impact of input error on the performance of the system with respect to the arrival rate $\lambda$ of the jobs, for different communication to computation ratios and input error limits. We ran 30 replications of the simulation for each set of input parameters, with different seeds of random numbers and for $10^4$ completed (that is, guaranteed) jobs in each replication. For every mean value, a 95% confidence interval was evaluated. The half-widths of all confidence intervals are less than 5% of their respective mean values. The simulation results are analyzed below.

### 4.4 Simulation Results and Discussion

The simulation results are shown in Figures 2, 3 and 4. It can be observed that for all values of the input error limit $IEL$ and all types of DAGs, the overall system performance $OSP$ decreases with the increase of the arrival rate $\lambda$. This is due to the heavier workload, which causes higher waiting times for the tasks in the queues. Therefore, more jobs miss their deadline. Moreover, the system exhibits the best performance when the DAGs are moderate (i.e. when $CCR = 1$), poorer performance when the DAGs are communication intensive (i.e. when $CCR = 10$) and the worst performance when the DAGs are computationally intensive (i.e. when $CCR = 0.1$). This suggests that, in our case study, the computational cost has a greater negative impact on the system performance than the communication cost of the jobs.

**Table 1.** Simulation Input Parameters

| Parameter description | Value |
|---|---|
| Number of processors in the system | $q = 64$ |
| Mean execution rate of processors | $\overline{\mu} = 1$ |
| Mean data trans. rate of comm. links | $\overline{\nu} = 1$ |
| Heterogeneity factor | $HF = 0.5$ |
| Maximum number of tasks in a job | $MAX = 64$ |
| Number of tasks in each job | $a \sim U[1, MAX]$ |
| Arrival rate of the jobs | $\lambda = \{0.1, 0.2, 0.3, 0.4\}$ |
| Relative deadline of each job | $RD \sim U[CPL, 2CPL]$ |
| CCR of the jobs | $CCR = \{0.1, 1, 10\}$ |
| Result precision threshold | $RPT = 0.75$ |
| Input error limit | $IEL = \{0, 1.5 \cdot 10^{200}\}$ and $IEL \sim U[1, u]$ |
| Mean comp. volume of the tasks | $\overline{w} = 10$ ($CCR = 0.1$) and |
| | $\overline{w} = 1$ ($CCR = \{1, 10\}$) |

In order to examine the impact of input error on the overall system performance $OSP$, we conducted three sets of experiments. In the first set of experiments, the input error limit of each task is considered to be equal to zero, i.e. $IEL = 0$ (Fig. 2). That is, no input error is allowed for any task, meaning that all parent tasks have to complete their whole optional part. In this case, only an exit task may discard a fraction of its optional part. As explained earlier, this may occur only when an exit task is the last task of its job remaining to be completed and the deadline of its job is reached. In the second set of experiments, the input error limit of each task is considered to be equal to a very large value ($IEL = 1.5 \cdot 10^{200}$) and thus considered infinite (Fig. 3). Therefore, all parent tasks can omit their whole optional part (provided that all the other conditions defined in subsection 2.1 are satisfied), as there is no limitation on the input error of their child tasks. In this case, it is assumed that by extending further the mandatory part of a child task, its input error can be corrected. In the third set of experiments, the input error limit is considered to be equal to a non-zero, finite value which may be different for each task in a job (Fig. 4). More specifically, we consider that the input error limit $IEL$ of each task is uniformly distributed in the range $[1, u]$, where $u$ is the number of parent tasks of the particular task. That is, in this case there is an upper limit on the input error allowed for each task, restricting the fraction of the optional part that each parent task can discard.

The simulation results show that when the input error limit is non-zero and finite for each task, we have the best overall system performance. When the input error limit is equal to zero for all tasks, the system exhibits poorer performance, whereas when the input error limit is considered infinite, we have the worst overall system performance. This may be explained by the fact that in condition (15), which must hold in order to skip the unprocessed part of the optional part of a parent task, we consider the average additional execution time imposed to each child task (over all processors in the system), which may be less than the actual

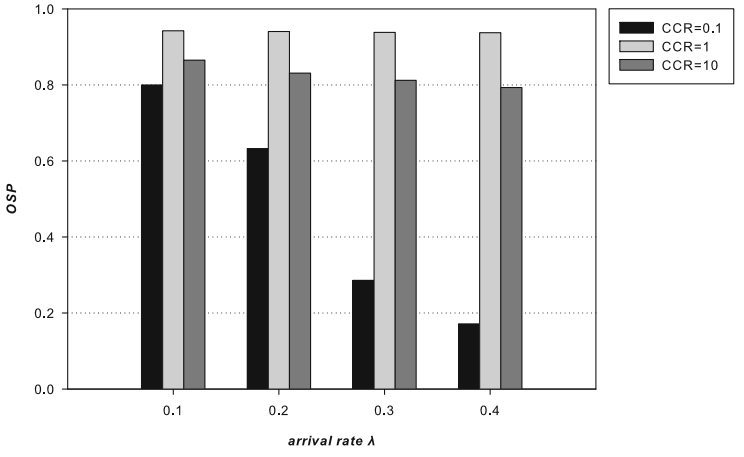



**Fig. 2.** $OSP$ vs. arrival rate $\lambda$, for task input error limit $IEL = 0$

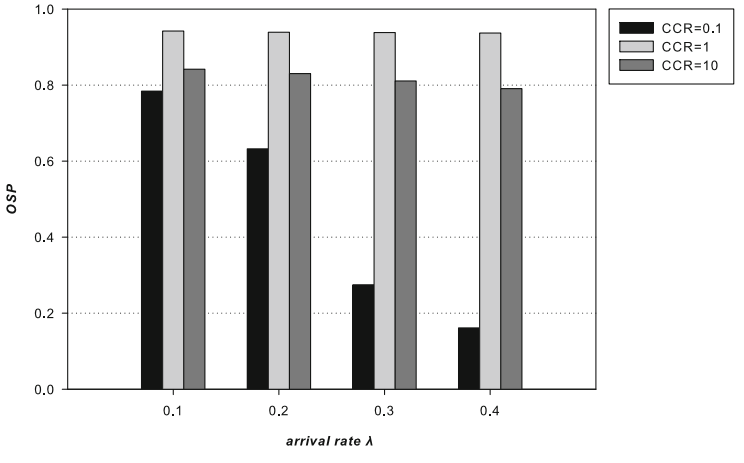



**Fig. 3.** $OSP$ vs. arrival rate $\lambda$, for task input error limit $IEL = 1.5 \cdot 10^{200}$

additional execution time of each child task (the actual imposed processing time of the child tasks cannot be calculated at the time instant the condition (15) is evaluated, since the child tasks are not yet allocated to processors). When the input error limit is considered infinite, more parent tasks omit their optional part and thus more child tasks extend their mandatory part. Therefore, there is a higher probability that the actual additional execution time of the child tasks is greater than their calculated average additional execution time and thus greater than the time saved by discarding the optional part of the parent tasks. This may result in delays, not only for the tasks of the particular job, but also for the

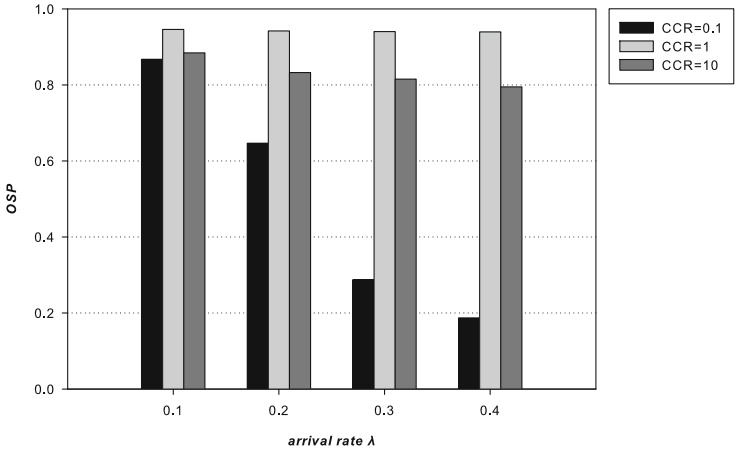



**Fig. 4.** $OSP$ vs. arrival rate $\lambda$, for task input error limit $IEL \sim U[1, u]$, where $u$ is the number of parent tasks of each task

tasks of other jobs in the system. Hence, more jobs may miss their deadline and the jobs that complete execution may have lower result precision. These factors lead to a poorer overall system performance.

On the other hand, when the input error limit is zero, parent tasks are not allowed to omit their optional part and thus no additional processing time is imposed to their child tasks. Therefore, the system performance is better compared to the case where the input error limit is considered infinite. Nevertheless, in the case where the input error limit is zero, processing time is never saved by skipping optional parts. Consequently, when the input error limit is non-zero and finite, it provides a moderate solution compared to the other two extreme cases and ultimately leads to a better system performance. In this case, parent tasks are allowed to discard their optional part, but in a restricted manner, so that there is a higher probability that the processing time saved is greater than the actual additional time imposed to the child tasks.

## 5 Conclusions and Future Work

In this paper, we investigated the impact of input error on the performance of a heterogeneous distributed real-time system, where task graphs with end-to-end deadlines are scheduled dynamically as they arrive in the system, utilizing imprecise computations. The performance of the system was evaluated by simulation, under various workloads and input error limits. The simulation results suggest that, in our case study, if input error is allowed in a limited way, it can lead to a better overall system performance, compared to the cases where the input error of a task is unlimited or equal to zero.

However, we could investigate the case where the structure of the task graphs in the system exhibits higher variability. Moreover, in the condition examined in order to skip the unprocessed part of the optional part of a parent task, the worst-case additional processing time imposed to the child tasks could be considered, instead of the average. Therefore, further research is required in order to address these issues.

## References


1. Buttazzo, G.C.: Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 2nd edn. Springer, New York (2004)
2. Lin, K.J., Natarajan, S., Liu, J.W.S.: Imprecise Results: Utilizing Partial Computations in Real-Time Systems. In: 8th IEEE Real-Time Systems Symposium, pp. 210–217 (1987)
3. Han, C.C., Shin, K.G., Wu, J.: A Fault-Tolerant Scheduling Algorithm for Real-Time Periodic Tasks with Possible Software Faults. IEEE Trans. Comp. 52(3), 362–372 (2003)
4. Hull, D., Shankar, A., Nahrstedt, K., Liu, J.W.S.: An End-to-End QoS Model and Management Architecture. In: IEEE Workshop on Middleware for Distributed Real-time Systems and Services, pp. 82–89 (1997)
5. Karatza, H.D.: Performance of Gang Scheduling Policies in the Presence of Critical Sporadic Jobs in Distributed Systems. In: 2007 International Symposium on Performance Evaluation of Computer and Telecommunication Systems, pp. 547–554 (2007)
6. Sha, L., Abdelzaher, T., Arzen, K.E., Cervin, A., Baker, T., Burns, A., Buttazzo, G., Caccamo, M., Lehoczky, J., Mok, A.K.: Real Time Scheduling Theory: A Historical Perspective. Real-Time Systems 28(2–3), 101–155 (2004)
7. Stavrinides, G.L., Karatza, H.D.: Scheduling Multiple Task Graphs with End-to-End Deadlines in Distributed Real-Time Systems Utilizing Imprecise Computations. Journal of Systems and Software 83(6), 1004–1014 (2010)
8. Liu, C.L., Layland, J.W.: Scheduling Algorithms For Multiprogramming in a Hard Real-Time Environment. Journal of the ACM 20(1), 46–61 (1973)
9. Topcuoglu, H., Hariri, S., Wu, M.Y.: Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. IEEE Trans. Par. Distr. Syst. 13(3), 260–274 (2002)
10. Cheng, B.C., Stoyenko, A.D., Marlowe, T.J., Baruah, S.K.: LSTF: A New Scheduling Policy for Complex Real-Time Tasks in Multiple Processor Systems. Automatica 33(5), 921–926 (1997)
11. Feng, W.C., Liu, J.W.S.: Algorithms for Scheduling Real-Time Tasks with Input Error and End-to-End Deadlines. IEEE Trans. Soft. Eng. 23(2), 93–106 (1997)
12. Haweet, W.A.E., Meligy, H.H.E., Salam, I.A.E.: Imprecise Computation Technique to Schedule AND/OR Tasks with Global End-to-End Deadline in Distributed Real-Time Systems. In: SICE 2003 Annual Conference, pp. 663–668 (2003)